

Proiectarea cu Microprocesoare
Curs 9

Andrei Voinescu

POLITEHNICA University of Bucharest

2021



Outline

Introducere

Memorii pe interfață paralelă



Context

- Până acum am studiat un microcontroller și posibilitățile de aplicații în jurul lui

- Până acum am studiat un microcontroller și posibilitățile de aplicații în jurul lui
- Nu toate problemele se rezolvă cu un sistem de calcul bazat pe microcontrollere, cu atât mai puțin pe unul de 8 biți



- Până acum am studiat un microcontroller și posibilitățile de aplicații în jurul lui
- Nu toate problemele se rezolvă cu un sistem de calcul bazat pe microcontrollere, cu atât mai puțin pe unul de 8 biți
- Microcontrollerul are memorie nonvolatilă, memorie RAM periferice într-un singur pachet pentru a reduce costurile unui sistem de calcul bazat pe el



- Până acum am studiat un microcontroller și posibilitățile de aplicații în jurul lui
- Nu toate problemele se rezolvă cu un sistem de calcul bazat pe microcontrollere, cu atât mai puțin pe unul de 8 biți
- Microcontrollerul are memorie nonvolatilă, memorie RAM periferice într-un singur pachet pentru a reduce costurile unui sistem de calcul bazat pe el
- La capătul opus al spectrului, un microprocesor conține doar capabilități de calcul și control, cu foarte puțină memorie



Context

- Până acum am studiat un microcontroller și posibilitățile de aplicații în jurul lui
- Nu toate problemele se rezolvă cu un sistem de calcul bazat pe microcontrollere, cu atât mai puțin pe unul de 8 biți
- Microcontrollerul are memorie nonvolatilă, memorie RAM periferice într-un singur pachet pentru a reduce costurile unui sistem de calcul bazat pe el
- La capătul opus al spectrului, un microprocesor conține doar capabilități de calcul și control, cu foarte puțină memorie
- Studiul unei plăci de bază contemporane este dificil datorită complexității și a informației non-publice



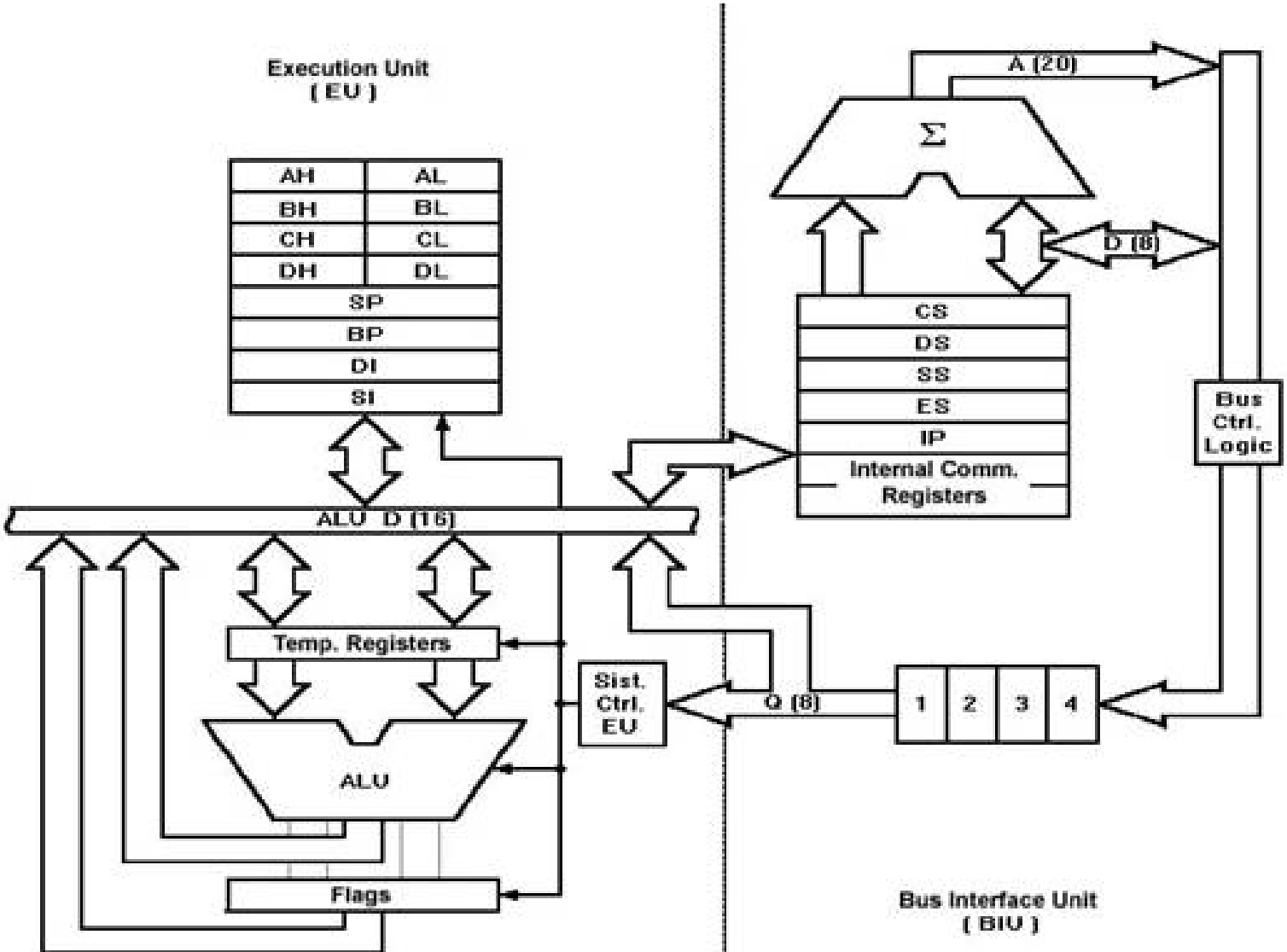
Context

- Până acum am studiat un microcontroller și posibilitățile de aplicații în jurul lui
- Nu toate problemele se rezolvă cu un sistem de calcul bazat pe microcontrollere, cu atât mai puțin pe unul de 8 biți
- Microcontrollerul are memorie nonvolatilă, memorie RAM periferice într-un singur pachet pentru a reduce costurile unui sistem de calcul bazat pe el
- La capătul opus al spectrului, un microprocesor conține doar capabilități de calcul și control, cu foarte puțină memorie
- Studiul unei plăci de bază contemporane este dificil datorită complexității și a informației non-publice
- Vom studia conceptele care au stat la baza plăcii cu 8086

- Dezvoltat în anul 1978
- Procesor de 16 biți cu 20 biți de adresă
- Magistrala de adrese multiplexată cu cea de date

- Dezvoltat în anul 1978
- Procesor de 16 biți cu 20 biți de adresă
- Magistrala de adrese multiplexată cu cea de date
- Pachet 40 pini - alimentare 5V - la fel ca ATMega324a
- 14 registre de 16 biți
- Cache de instrucțiuni cu 6 bytes
- Două moduri de interfațare externă (ne uităm doar la unul)




Execution Unit
(EU)
AH
AL
BH
BL
CH
CL
DH
DL
SP
BP
DI
SI
ALU D (16)
Temp. Registers
ALU
Flags
Sist. Ctrl. EU
Q (8)
A (20)
Σ
D (8)
CS
DS
SS
ES
IP
Internal Comm. Registers
Bus Ctrl. Logic
1
2
3
4
Bus Interface Unit
(BIU)

- Arhitectură CISC $\rightarrow$ registrele au funcții speciale

- Arhitectură CISC $\rightarrow$ registrele au funcții speciale
- Registre acumulatori - destinații + surse automate în unele instrucțiuni

- Arhitectură CISC $\rightarrow$ registrele au funcții speciale
- Registre acumulatori - destinații + surse automate în unele instrucțiuni
- Registre index - folosite ca index automat în anumite instrucțiuni (ex. instrucțiuni stringuri)

- Arhitectură CISC $\rightarrow$ registrele au funcții speciale
- Registre acumulatori - destinații + surse automate în unele instrucțiuni
- Registre index - folosite ca index automat în anumite instrucțiuni (ex. instrucțiuni stringuri)
- Registre segment - identifică **segmente**

- Arhitectură CISC $\rightarrow$ registrele au funcții speciale
- Registre acumulatori - destinații + surse automate în unele instrucțiuni
- Registre index - folosite ca index automat în anumite instrucțiuni (ex. instrucțiuni stringuri)
- Registre segment - identifică **segmente**
  - **segmentele** sunt bucăți de 64kB referențiate de anumite instrucțiuni



Registre (II)

- `AX (AH|AL)` - Acumulator principal
- `BX (BH|BL)` - Acumulator și bază
- `CX (CH|CL)` - Acumulator/Numărător
- `DX (DH|DL)` - Acumulator/Numărător/Date IO



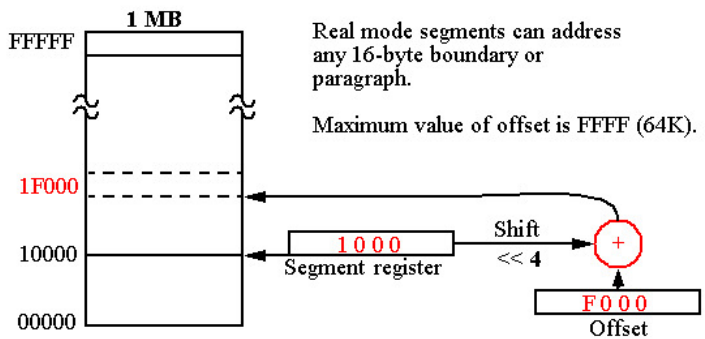
Registre (II)

- `AX (AH|AL)` - Acumulator principal
- `BX (BH|BL)` - Acumulator și bază
- `CX (CH|CL)` - Acumulator/Numărător
- `DX (DH|DL)` - Acumulator/Numărător/Date IO
- `SP` - Stack pointer
- `BP` - Base pointer
- `SI` - Source index
- `DI` - Destination index

- CS - Code Segment
- DS - Data Segment
- SS - Stack Segment
- ES - Extended Segment

- `CS` - Code Segment
- `DS` - Data Segment
- `SS` - Stack Segment
- `ES` - Extended Segment
- `IP` - Program Counter/ Instruction Pointer
- `FLAGS` - Registrul cu flags

- Segmentele sunt:
  - CS:IP
  - SS:SP
  - DS:DI
  - ES:SI



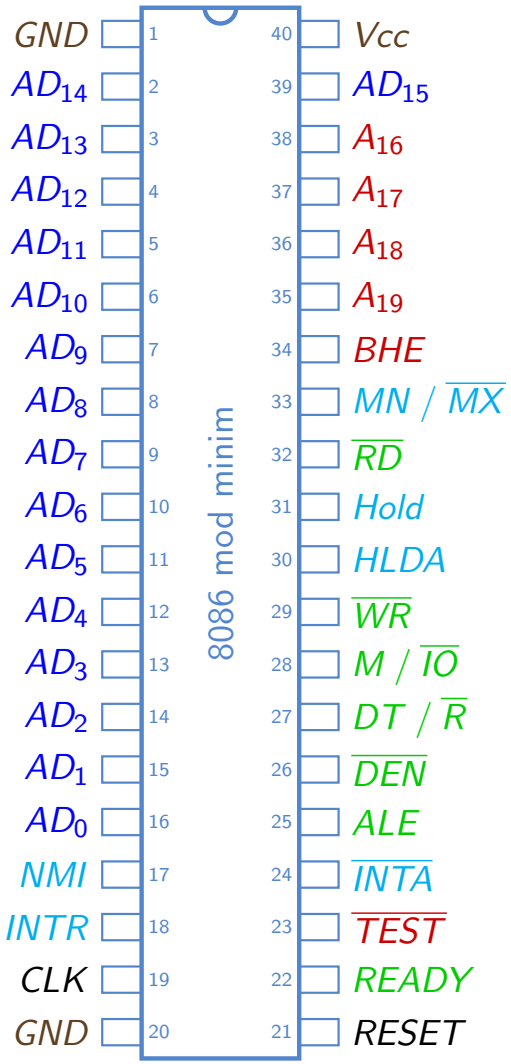
- Adresare directă - `MOV AX,BX`
- Adresare indirectă de registru - `MOV AX,[BX]`
- Bazată sau indexată - se folosește (SS, SI, DI)
- Bazată și indexată - și index și bază
- Toate Variantele de adresare pot avea și offset imediat (mai puțin cea directă)



Flags

- Carry - carry/borrow la cel mai semnificativ bit
- Parity - 1 dacă suma biților este pară
- Zero - 1 dacă rezultatul este 0
- Sign - bitul cel mai semnificativ al rezultatului
- Direction - 1 dacă se face auto-decrement
- Interrupt Enable - 1 dacă întreruperile mascabile sunt activate
- Trap/Single step - dacă e 1 întreruperea de single-step va apărea după următoarea instrucțiune
- Overflow
- Auxiliary - carry/borrow pentru biții 0-3

Pinout


8086 mod minim
GND 1
AD14 2
AD13 3
AD12 4
AD11 5
AD10 6
AD9 7
AD8 8
AD7 9
AD6 10
AD5 11
AD4 12
AD3 13
AD2 14
AD1 15
AD0 16
NMI 17
INTR 18
CLK 19
GND 20
40 Vcc
39 AD15
38 A16
37 A17
36 A18
35 A19
34 BHE
33 MN / MX
32 RD
31 Hold
30 HLDA
29 WR
28 M / IO
27 DT / R
26 DEN
25 ALE
24 INTA
23 TEST
22 READY
21 RESET

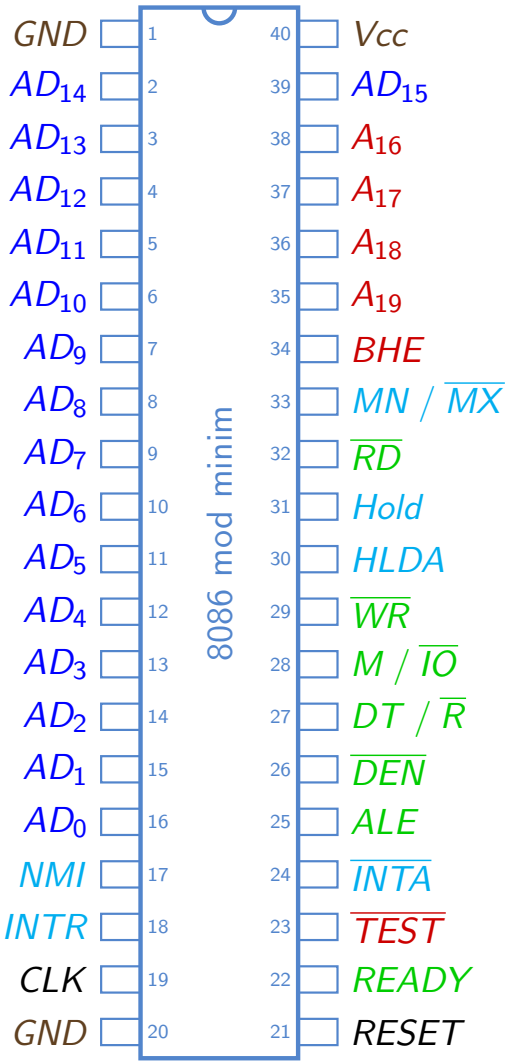
- Procesorul are două moduri, pinii pot avea funcționalități diferite între cele două



- Procesorul are două moduri, pinii pot avea funcționalități diferite între cele două
- Modul este selectabil hardware, printr-un pin

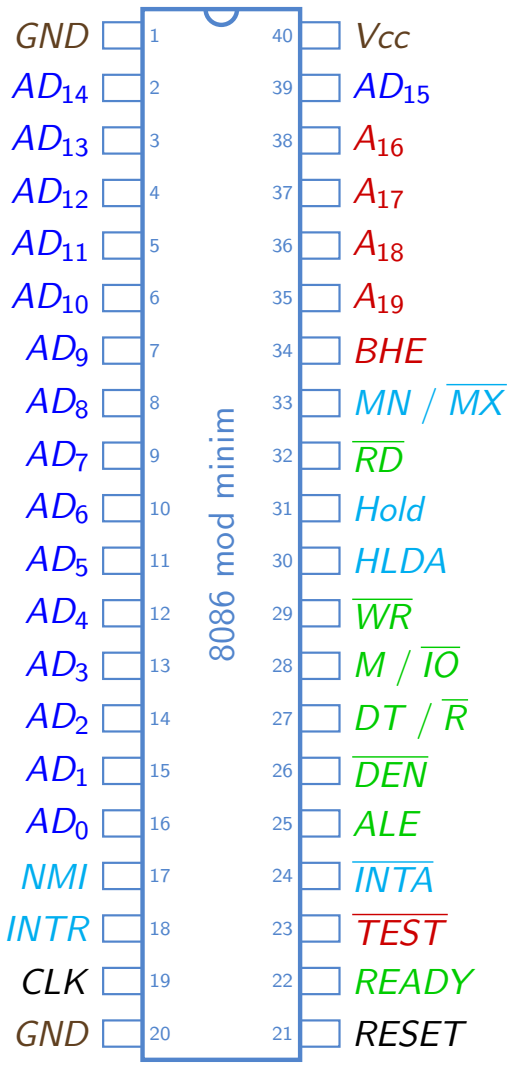
- Procesorul are două moduri, pinii pot avea funcționalități diferite între cele două
- Modul este selectabil hardware, printr-un pin
- Diferența constă în felul în care se accesează magistrală (ce fel de semnale generează procesorul)

Pinout (I)



- *(I/O/Z)* `AD15..0` Address/Data Bus
- *(O/Z)* `A19/S6..A16/S3` Address
- *(O)* $\overline{BHE}/S7$ Byte High Enable
- *(I)* $MN/\overline{MX}$ Minimum/Maximum Mode
- *(O/Z)* $M/\overline{IO}$ acces memorie sau I/O
- *(O)* $\overline{RD}$ Semnal read
- *(O)* $\overline{WR}$ Semnal write
- *(O)* $\overline{ALE}$ Address latch
- *(I)* $\overline{TEST}$ instrucțiunea wait $\leftarrow$ 0 pe pin
- *(I)* `READY` semnal de la alt dispozitiv

Pinout (I)



- *(I)* `INTR` apariția unei întreruperi
  - Verificată la sfârșitul ultimului ciclu al unei instrucțiuni
  - Ajungem la tabela de vectori
- *(I)* `NMI` întrerupere non-mascabilă, edge-triggered
- *(I)* `RESET` 4 cicli $\rightarrow$ execuție 0xFFFF0
- *(I)* `CLK` 33% duty cycle (2-8 MHz)
- *(O)* $\overline{INTA}$ interrupt ack
- *(O/Z)* $DT/\overline{R}$ Data Transmit/Receive
- *(O/Z)* $\overline{DEN}$ Data enable
- *(I)* `HOLD` cineva cere hold pe magistrală
- *(O)* `HOLDA` ack hold

- Microprocesorul extrage cod prin cicli de magistrală
  - Grupuri de perioade de ceas fără legătură cu instrucțiunile

- Microprocesorul extrage cod prin cicli de magistrală
  - Grupuri de perioade de ceas fără legătură cu instrucțiunile
- **T1** - MPU pune adresele + ALE, pe $ALE \downarrow$ se garantează adresele

- Microprocesorul extrage cod prin cicli de magistrală
  - Grupuri de perioade de ceas fără legătură cu instrucțiunile
- **T1** - MPU pune adresele + ALE, pe $ALE \downarrow$ se garantează adresele
- **T2** - MPU pregătește transferul de date ($S7..4, \overline{RD}, \overline{WR}$)

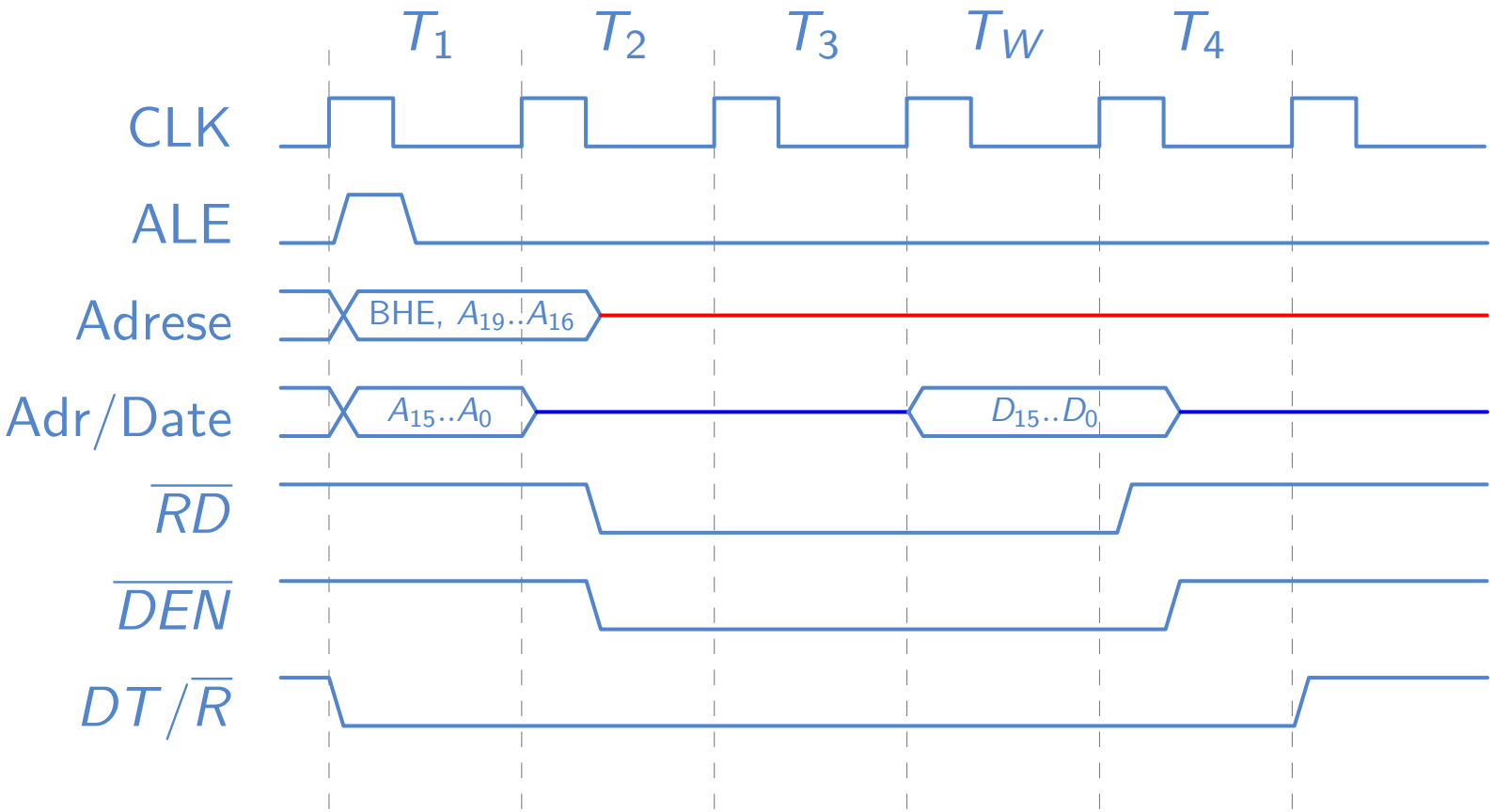
- Microprocesorul extrage cod prin cicli de magistrală
  - Grupuri de perioade de ceas fără legătură cu instrucțiunile
- **T1** - MPU pune adresele + ALE, pe $ALE \downarrow$ se garantează adresele
- **T2** - MPU pregătește transferul de date ($S7..4, \overline{RD}, \overline{WR}$)
- **T3** - MPU scrie sau citește, citirea poate fi extinsă prin READY=0 (**TW**)

- Microprocesorul extrage cod prin cicli de magistrală
  - Grupuri de perioade de ceas fără legătură cu instrucțiunile
- **T1** - MPU pune adresele + ALE, pe $ALE \downarrow$ se garantează adresele
- **T2** - MPU pregătește transferul de date ($S7..4, \overline{RD}, \overline{WR}$)
- **T3** - MPU scrie sau citește, citirea poate fi extinsă prin READY=0 (**TW**)
- **T4** - dezactivarea comenzilor și eliberarea magistralei locale

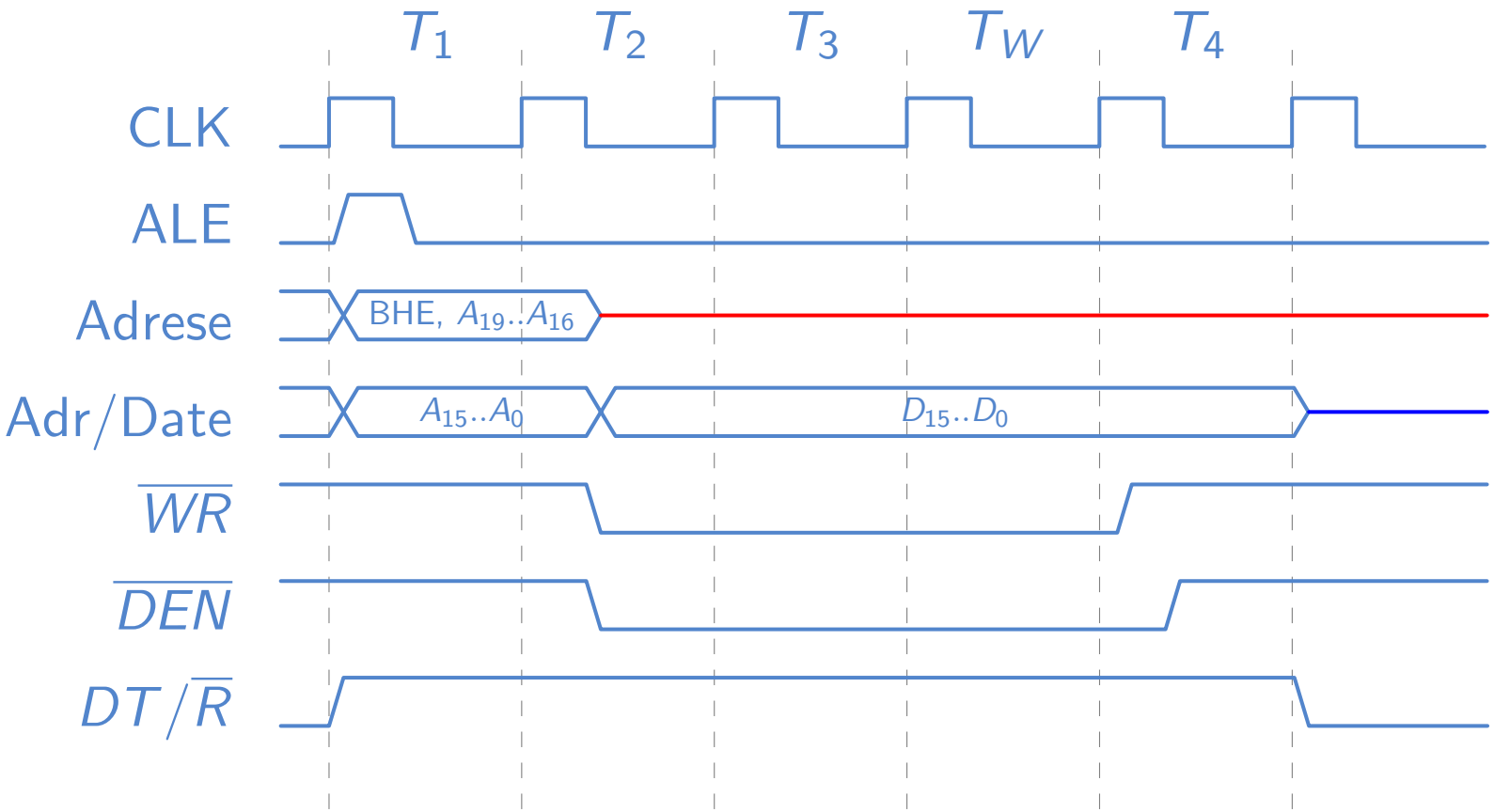
- MPU accesează magistrală cu ajutorul unor semnale

$M/\overline{IO}$	$\overline{RD}$	$\overline{WR}$	Transfer
0	0	1	Citire I/O
0	1	0	Scriere I/O
1	0	1	Citire memorie
1	1	0	Scriere memorie

Accesul la magistrala externă - Citirea

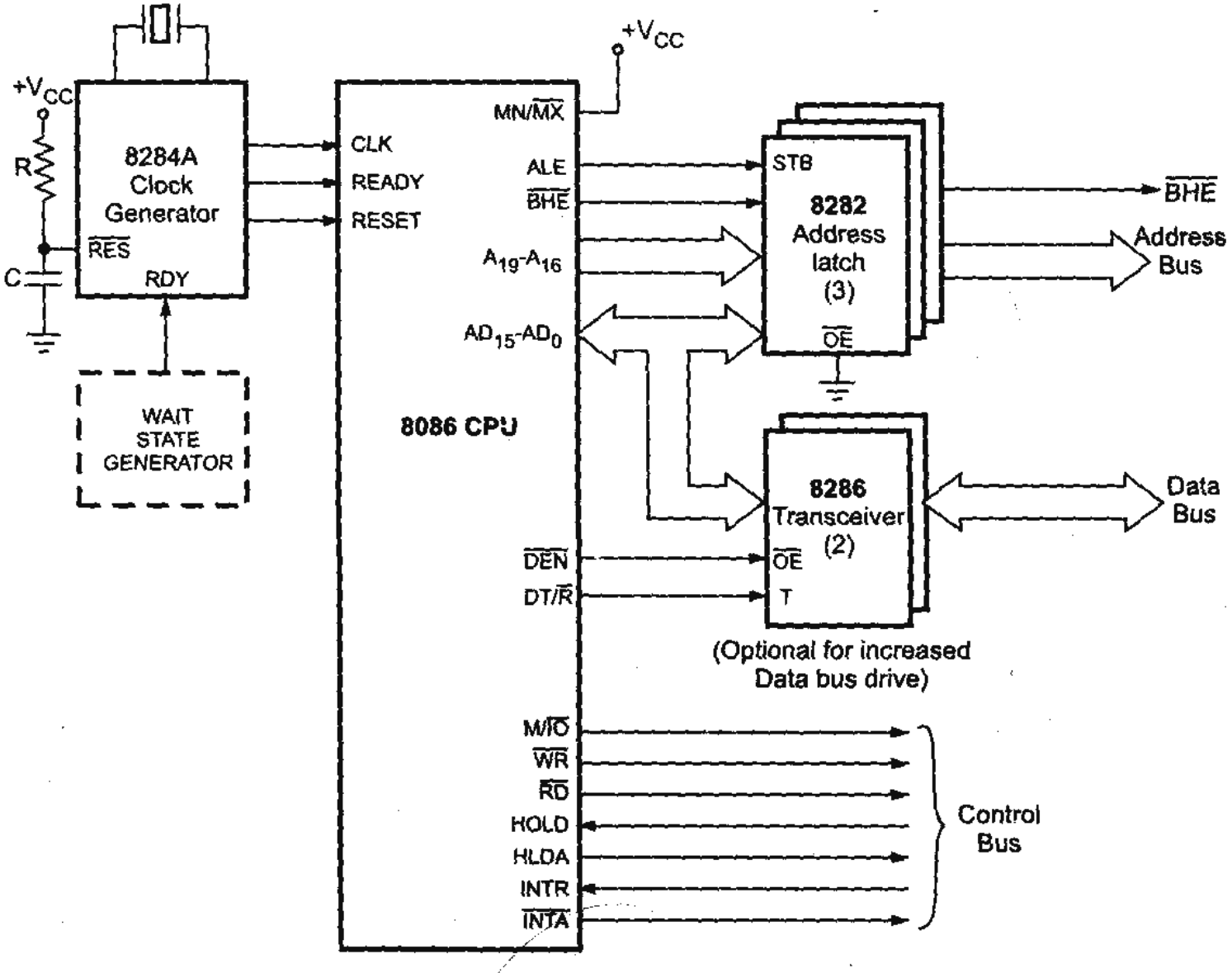


Accesul la magistrala externă - Scrierea



- Magistrala de date este de 16 biți, dar putem scrie și câte 8 la un moment dat.
- Adresele noastre accesează câte un octet

$\overline{BHE}$	$A0$	tip transfer
0	0	16 biți de la adresă pară
0	1	8 biți semnificativi de la adresă impară
1	0	8 biți nesemnificativi de la adresă pară
1	1	nimic


+VCC
R
C
8284A
Clock
Generator
RES
RDY
WAIT
STATE
GENERATOR
CLK
READY
RESET
MN/MX
ALE
BHE
A19-A16
AD15-AD0
8086 CPU
DEN
DT/R
M/IO
WR
RD
HOLD
HLDA
INTR
INTA
STB
8282
Address
latch
(3)
OE
BHE
Address
Bus
8286
Transceiver
(2)
OE
T
Data
Bus
(Optional for increased
Data bus drive)
Control
Bus

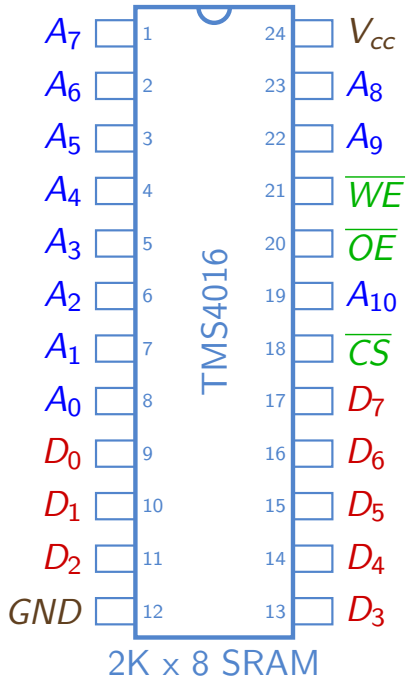


Introducere

Memorii pe interfață paralelă

Pinout general

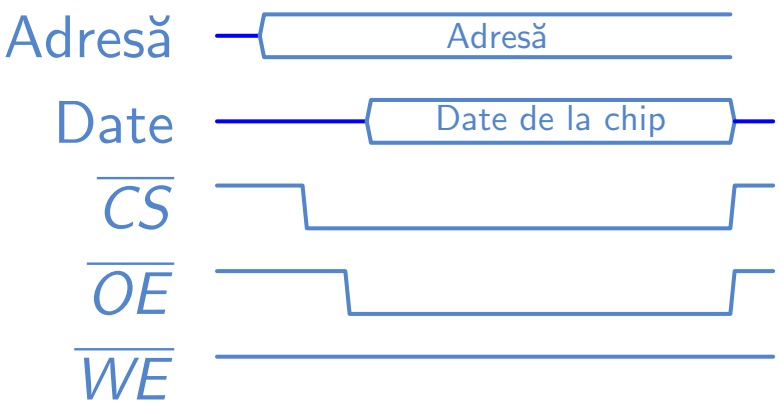
- Pini de adresă: Pe acești pini un alt dispozitiv codifică locația cuvântului de memorie dorit (N biți de adresă codifică 2^N locații
- Pini de date: La o locație sunt stocați un anumit număr de biți
- CS: pinul care selectează chip-ului. Nu se poate comunica decât cu un chip selectat

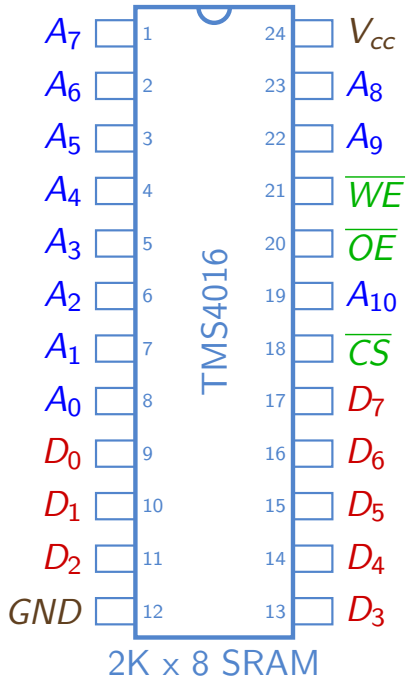


Pin(s)	Type	Function
$A_{10}..A_0$	I	Address
$D_7..D_0$	I/O	Data In/Out
$\overline{CE}$	I	Chip Enable
$\overline{OE}$	I	Output Enable
$\overline{WE}$	I	Write Enable

Pinout general

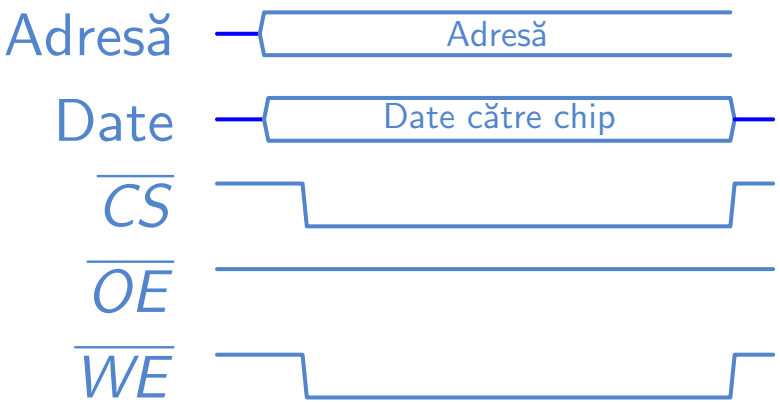
- `OE` - Output Enable: reflectă faptul că se dorește citirea din chip, iar chipului îi este permis să scrie pe magistrala de date
- Adresele sunt puse înainte de OE
- Datele sunt primite la Δt după OE

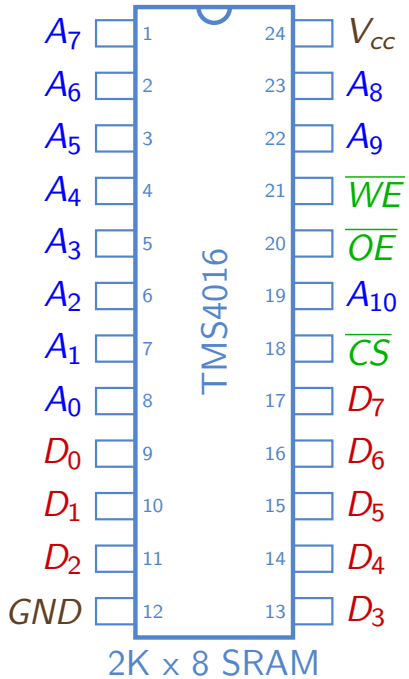




Pin(s)	Type	Function
$A_{10}..A_0$	I	Address
$D_7..D_0$	I/O	Data In/Out
$\overline{CE}$	I	Chip Enable
$\overline{OE}$	I	Output Enable
$\overline{WE}$	I	Write Enable

- `WE` - Write Enable: reflectă faptul că se dorește să se scrie în locația de memorie
- Adresele și datele sunt puse înainte de WE
- Este lăsat un timp după WE

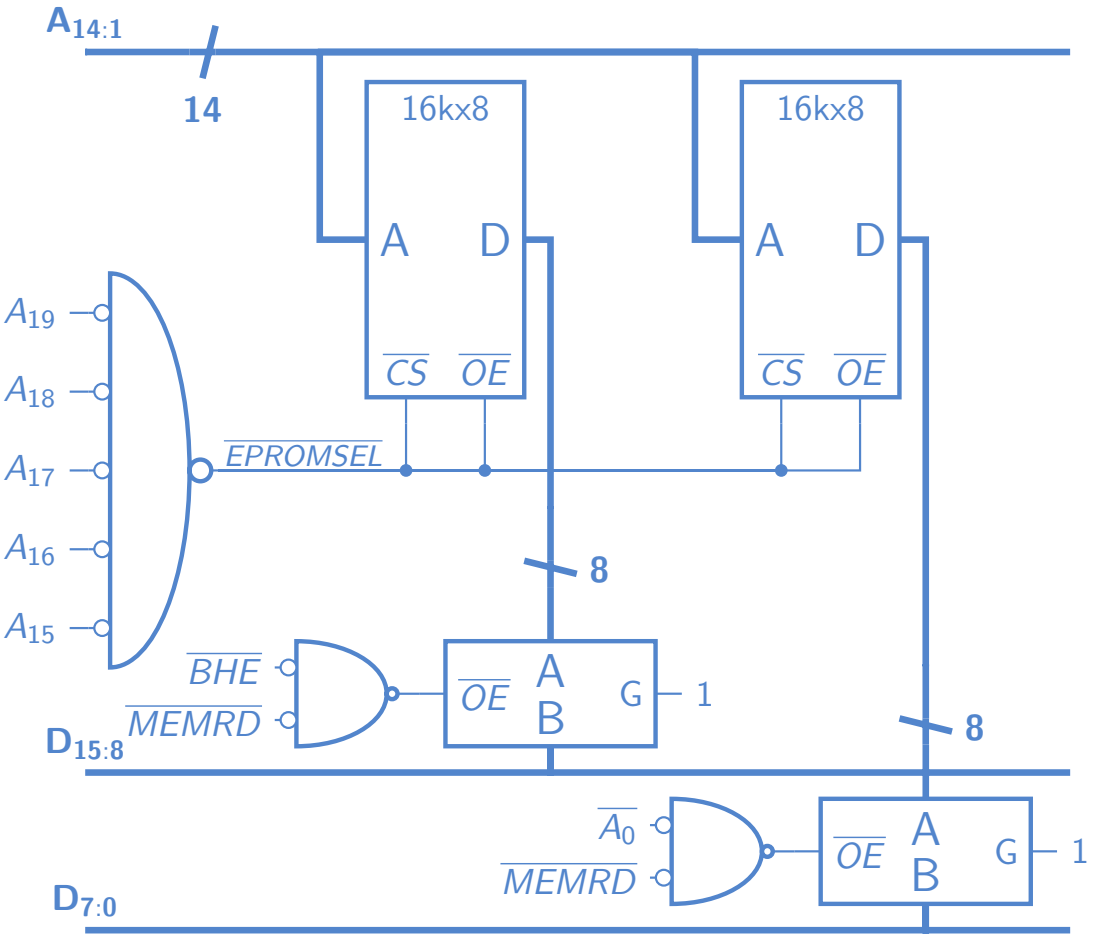




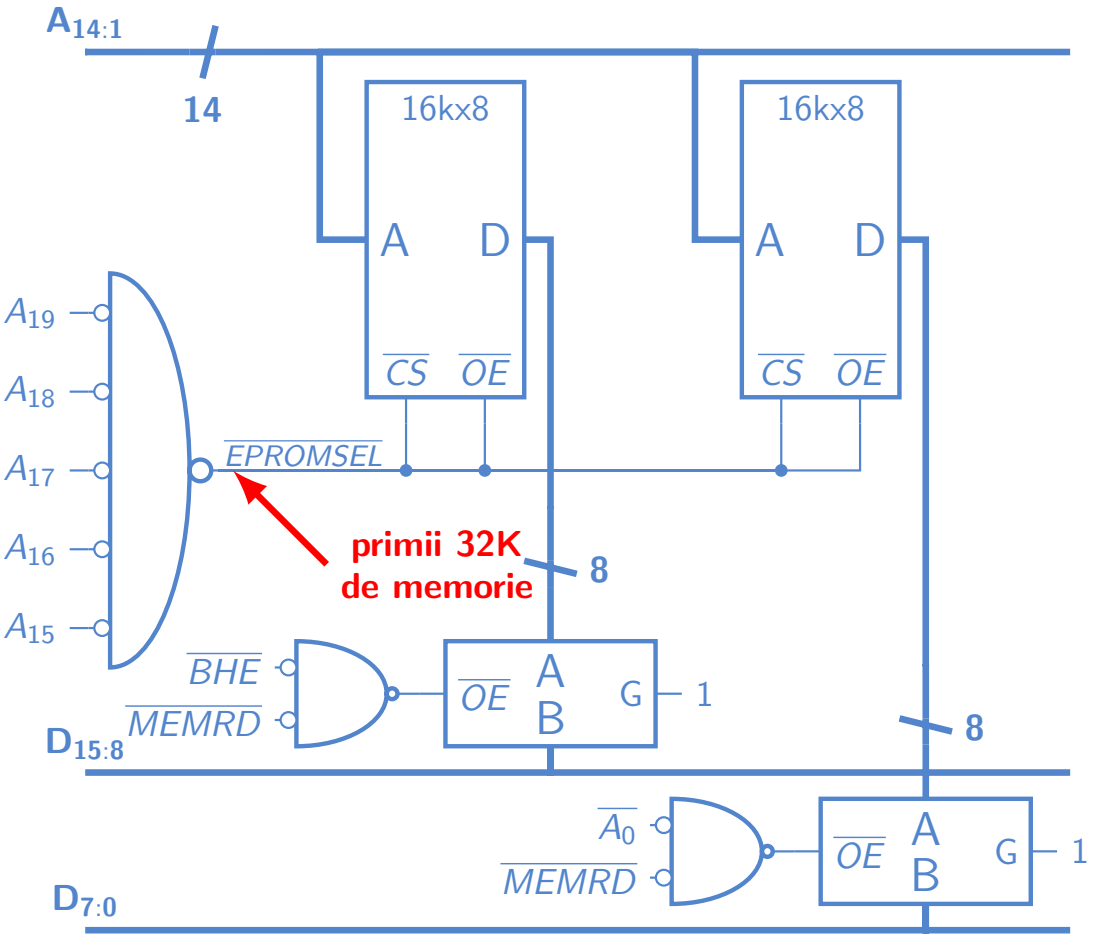
Pin(s)	Type	Function
$A_{10}..A_0$	I	Address
$D_7..D_0$	I/O	Data In/Out
$\overline{CE}$	I	Chip Enable
$\overline{OE}$	I	Output Enable
$\overline{WE}$	I	Write Enable

- 8086 are magistrala pe 16 biți
  - Conectăm câte două chip-uri x8 în paralel
  - Folosim ultimul bit de adresă A_0 pentru a selecta între ele
- 8086 are magistrala pe 16 biți

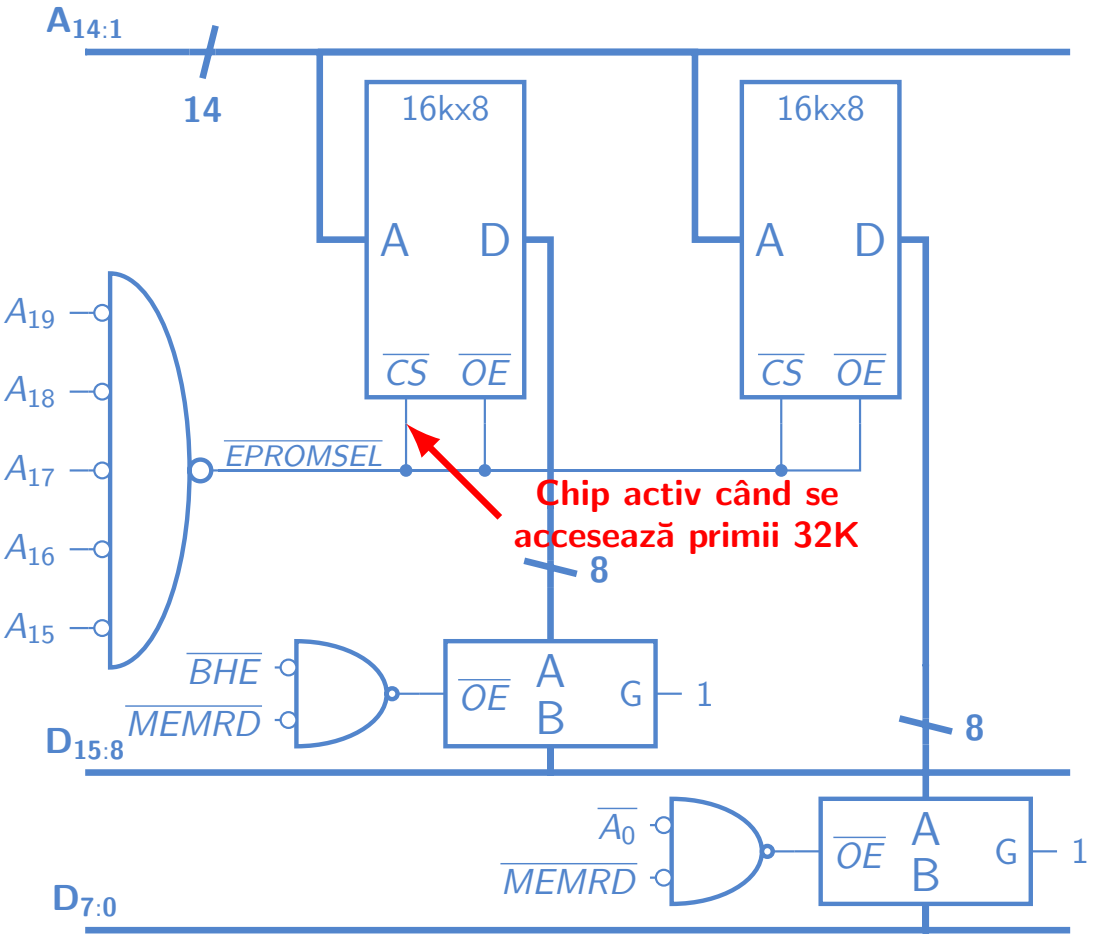
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000-0x07FFF



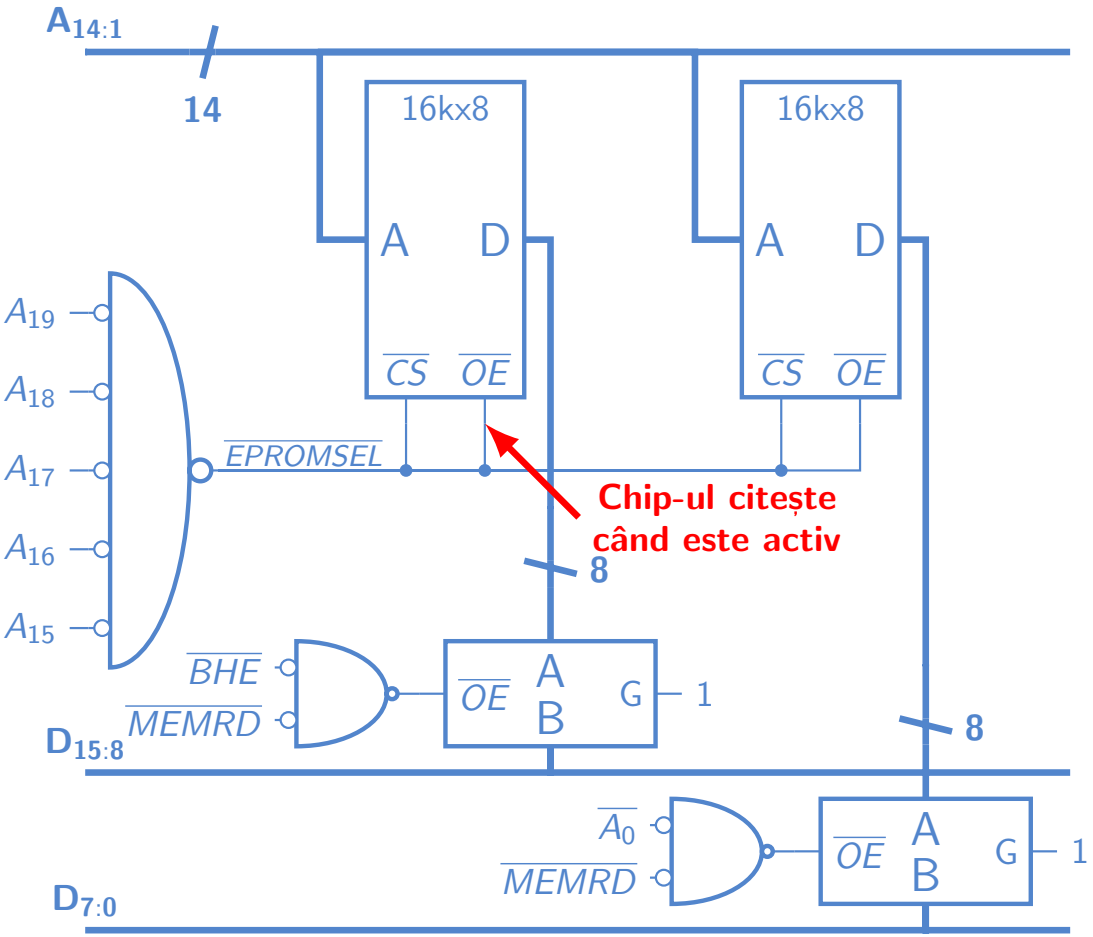
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000–0x07FFF



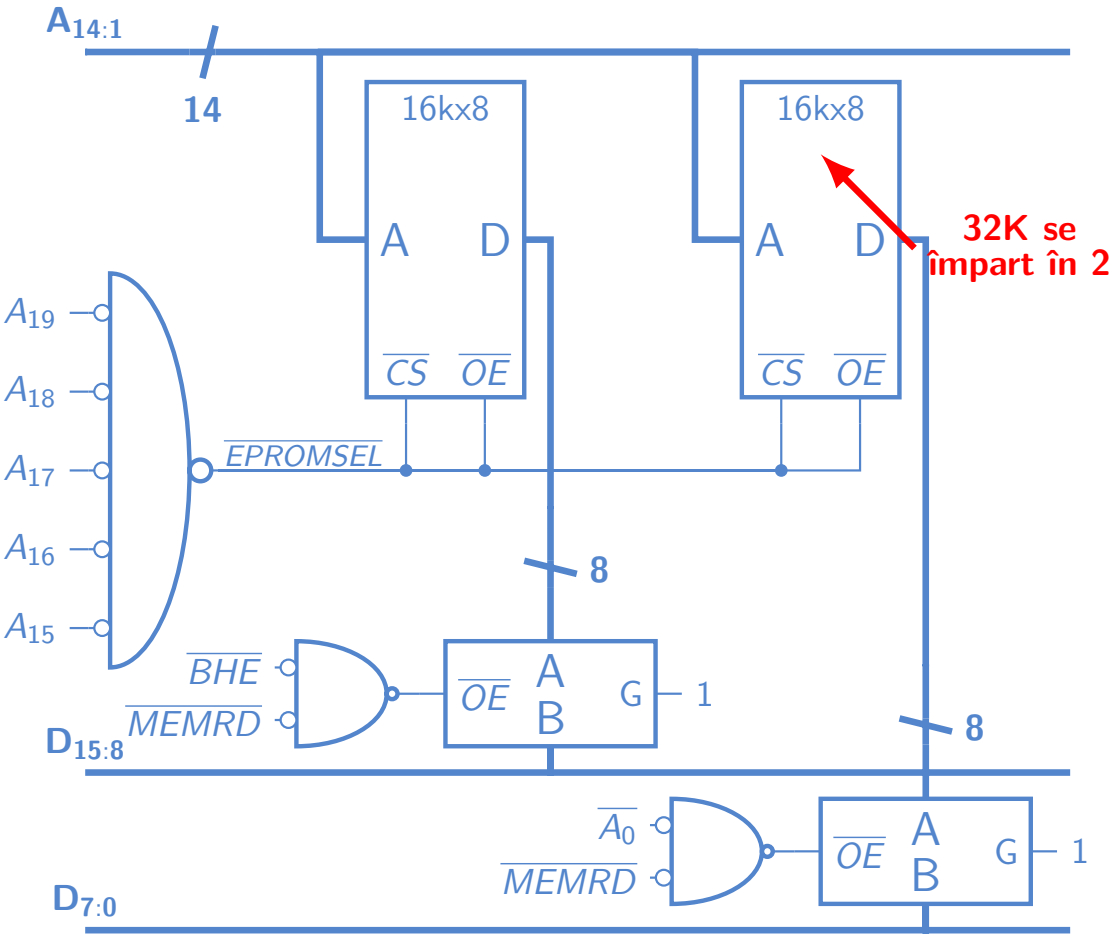
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000-0x07FFF



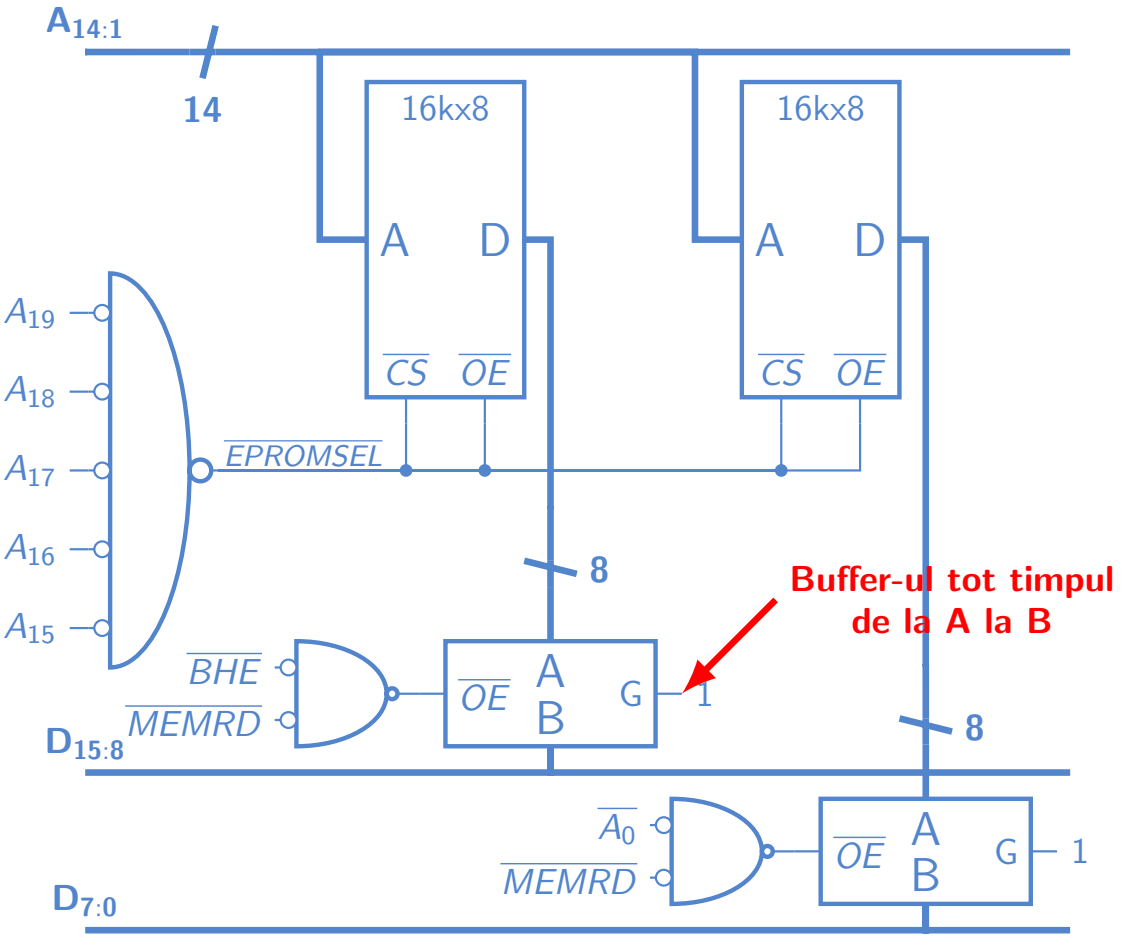
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000–0x07FFF



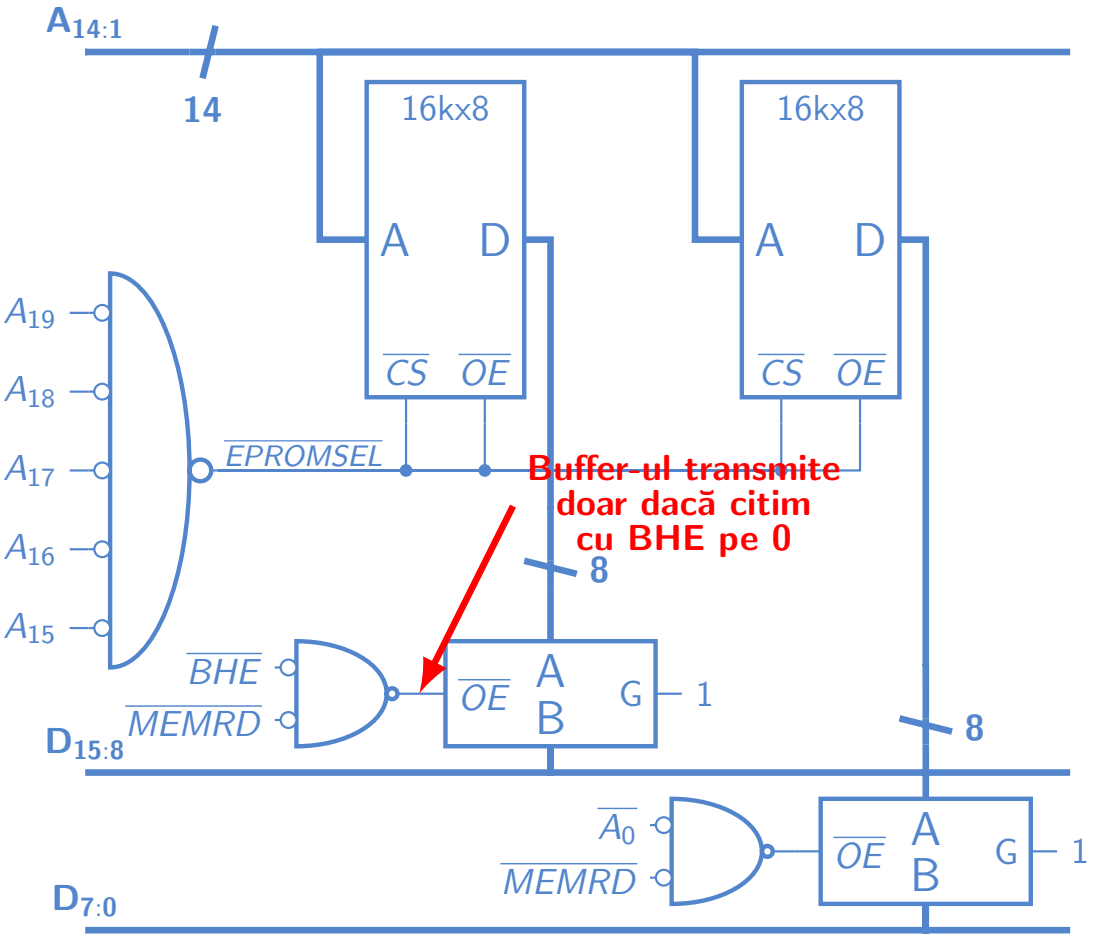
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000-0x07FFF



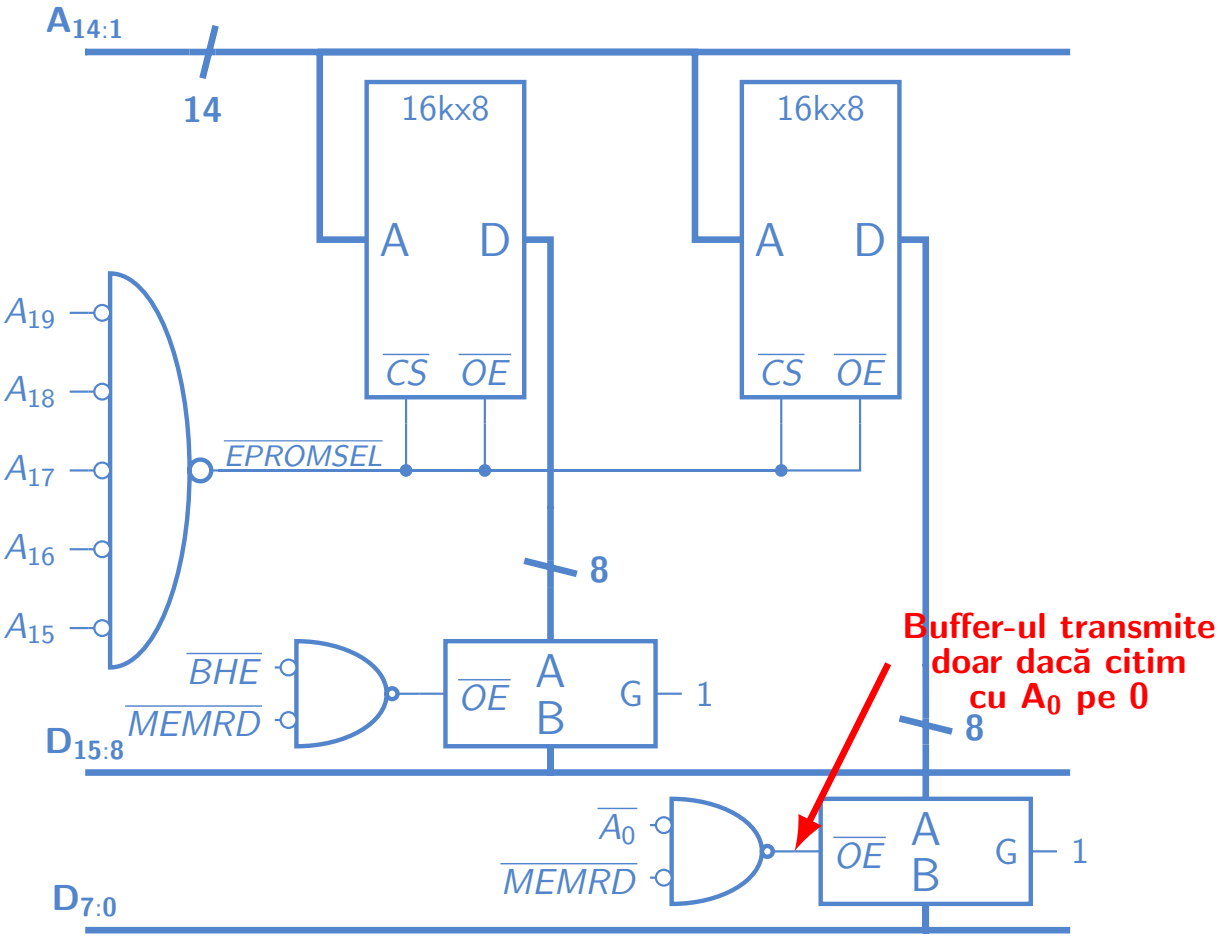
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000-0x07FFF



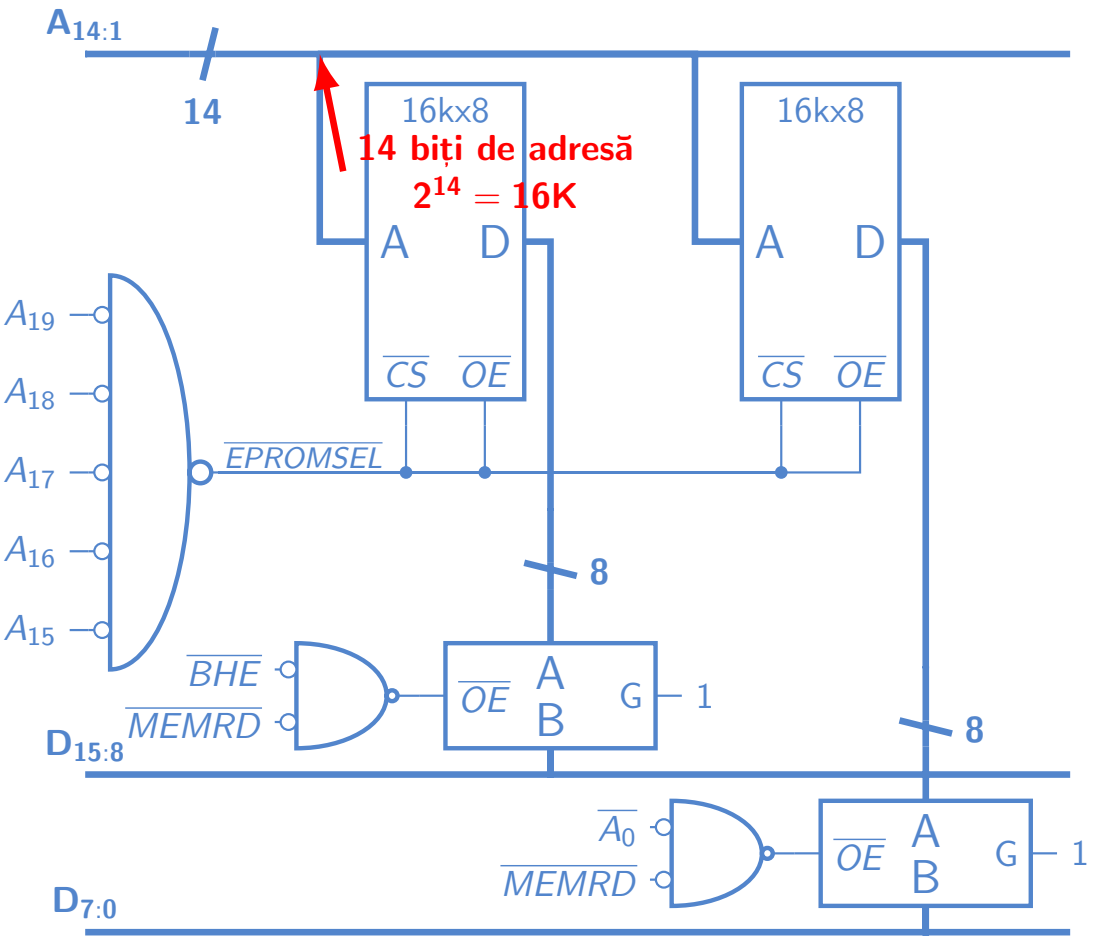
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000-0x07FFF



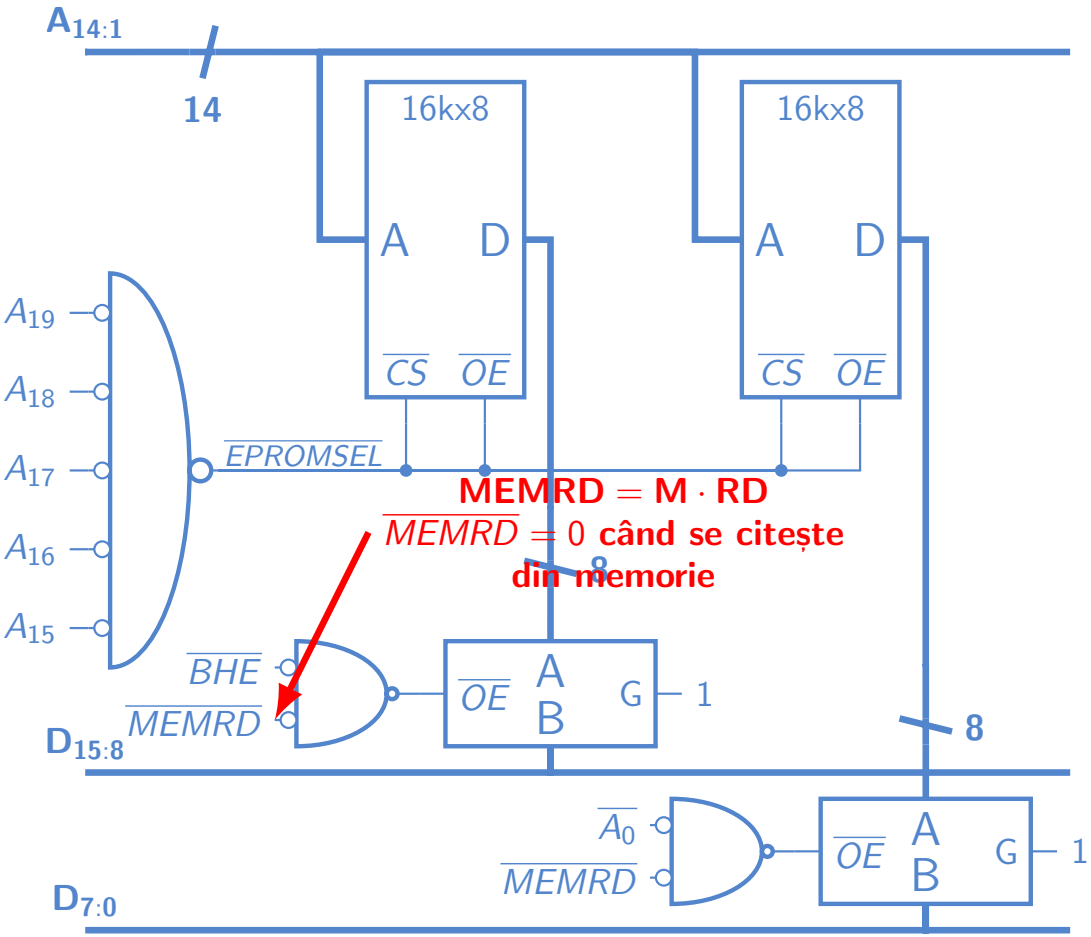
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000–0x07FFF



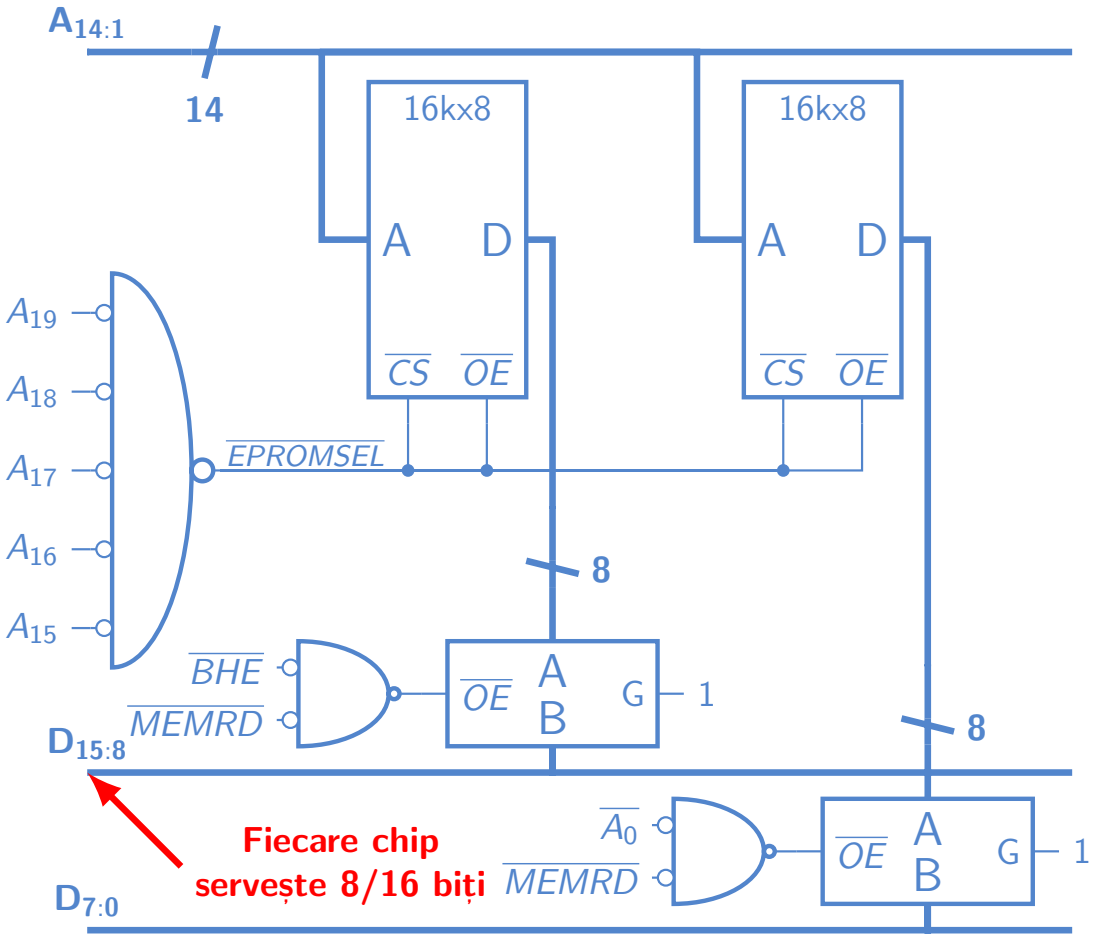
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000–0x07FFF



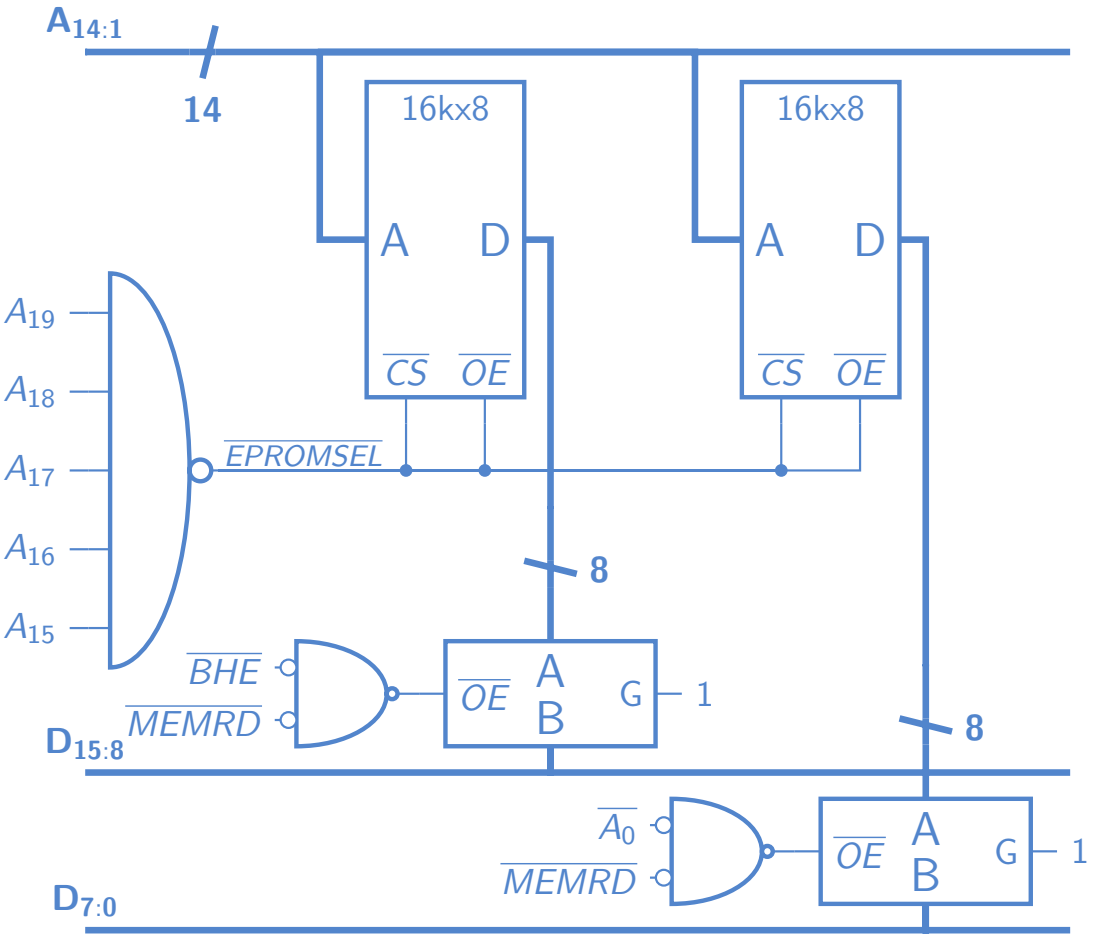
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000–0x07FFF



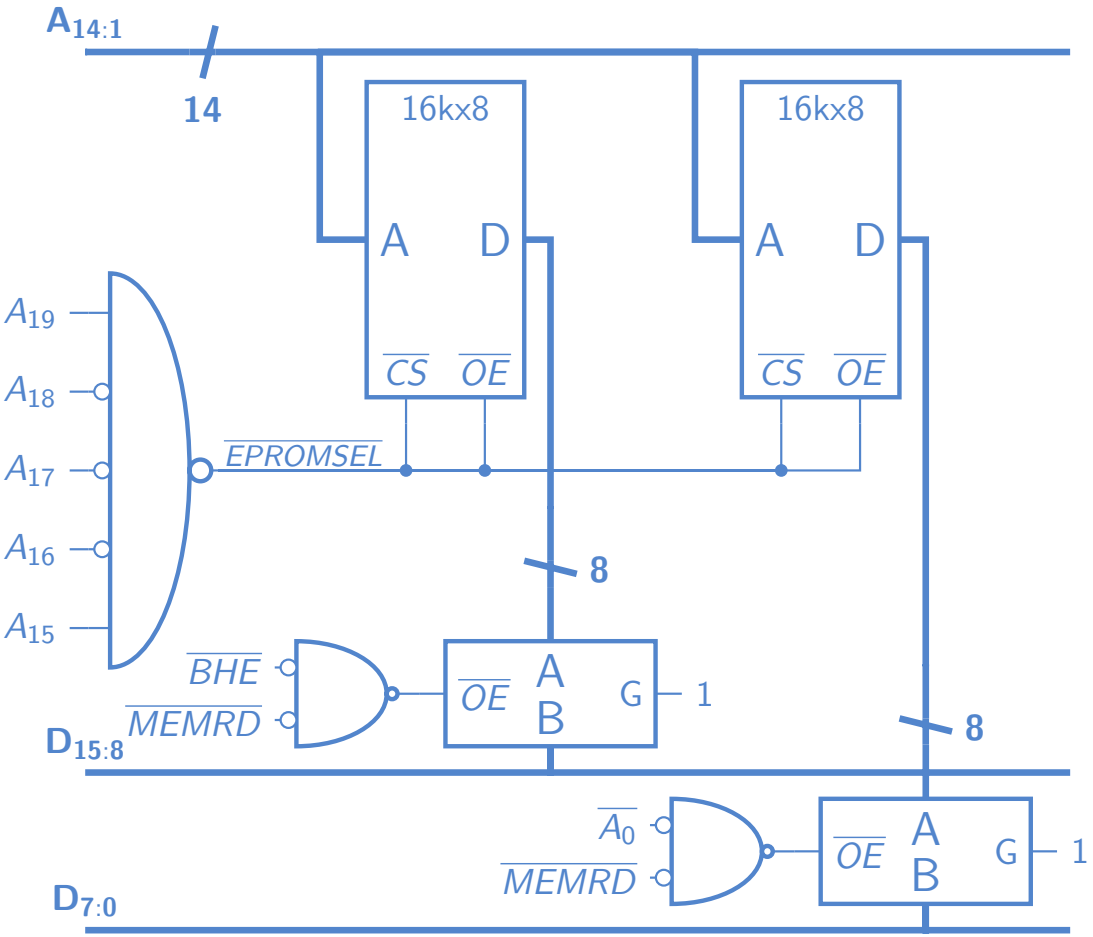
- Mapăm în primii 32K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0x00000–0x07FFF



- Mapăm în **ultimii** 32K EPROM (read-only)
- Cu Chip-uri x8
- $EPROMSEL = A_{19} \cdot A_{18} \cdot A_{17} \cdot A_{16} \cdot A_{15}$
- $\overline{EPROMSEL}$ activ pe 0
- Range 0xF8000–0xFFFFF



- Mapăm 32K EPROM (read-only) de la adresa 0x88000
- $\overline{EPROMSEL} = A_{19} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot A_{15}$
- Range 0x88000–0x8FFFF



- Dar dacă nu vrem de la o adresă multiplu de 32K?



32K de la orice adresă

- Dar dacă nu vrem de la o adresă multiplu de 32K?
- EPROMSEL va deveni mai complicat



32K de la orice adresă

- Dar dacă nu vrem de la o adresă multiplu de 32K?
- EPROMSEL va deveni mai complicat
- EPROMSEL este un "factor comun" pentru toate locațiile
- Memoriile vor face wrap-around intern

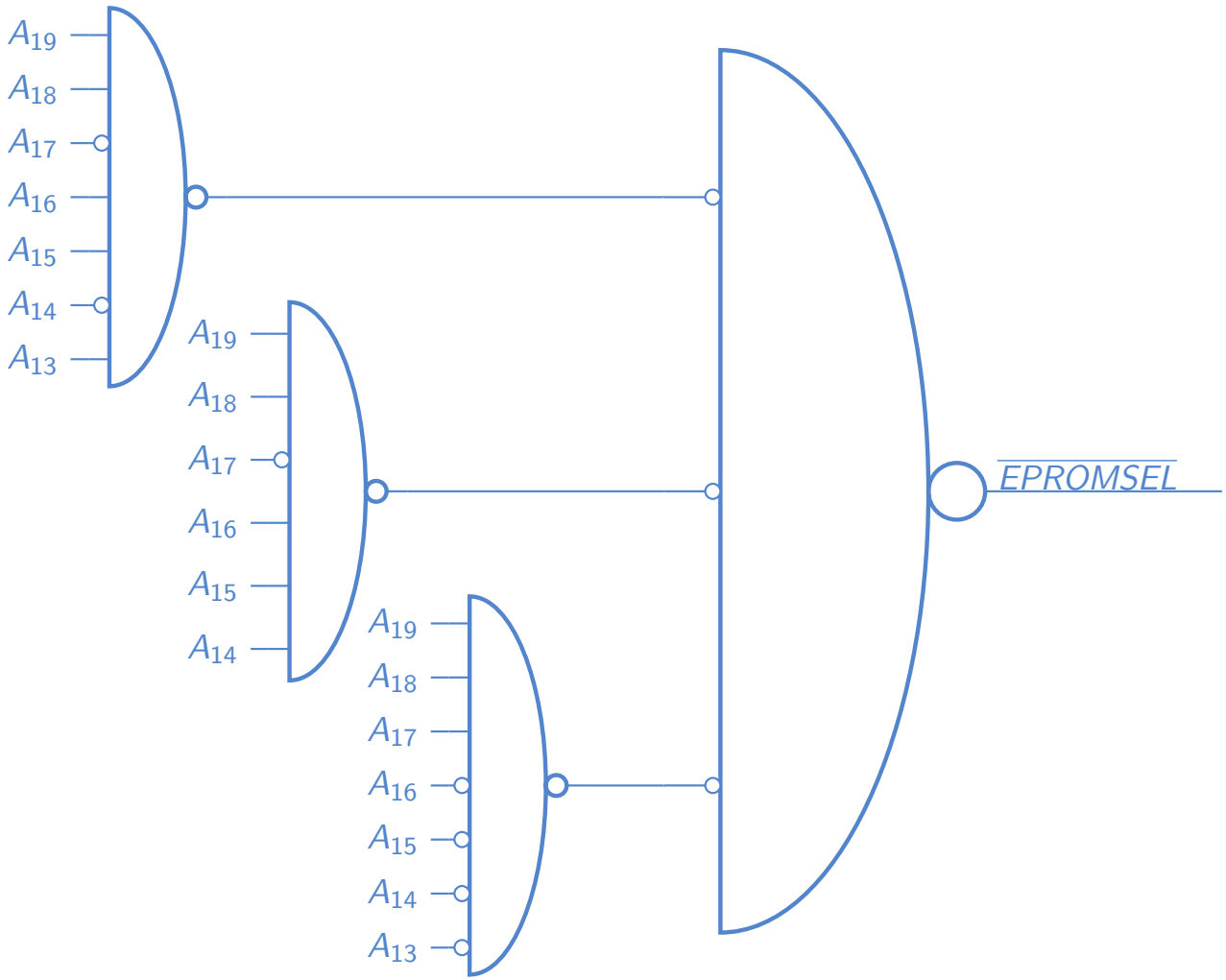
- Dar dacă nu vrem de la o adresă multiplu de 32K?
- EPROMSEL va deveni mai complicat
- EPROMSEL este un "factor comun" pentru toate locațiile
- Memoriile vor face wrap-around intern
- Ex: 32K de la `0xDA000`

$$0xDA000 + 2^{15} - 1 = 0xE1FFF$$

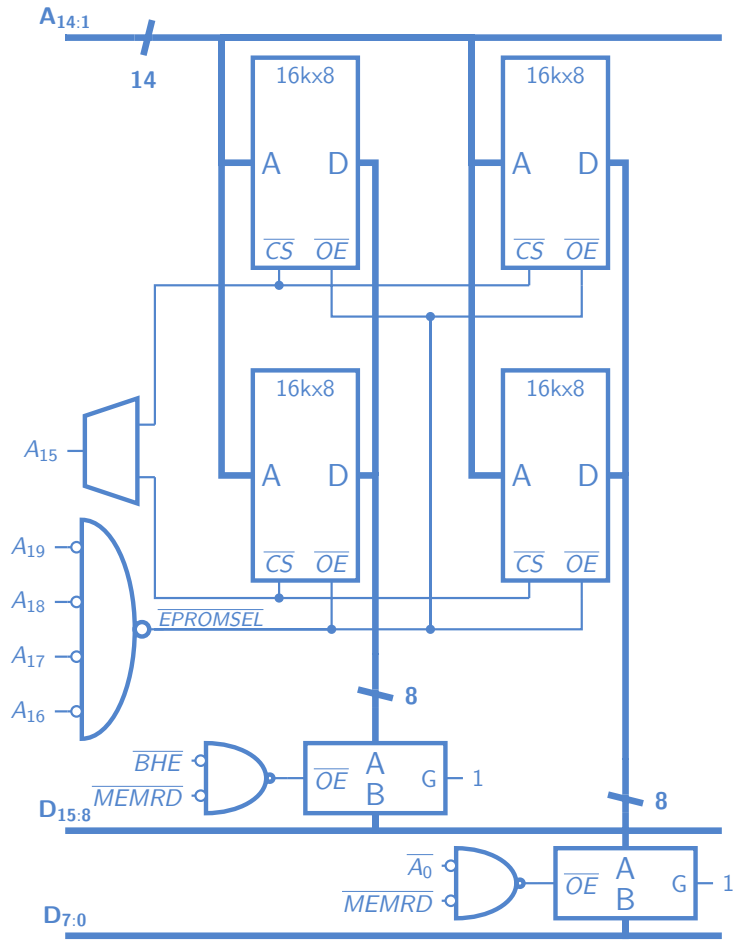
$$0xDA000 \rightarrow 0xDBFFF = A_{19} \cdot A_{18} \cdot \overline{A_{17}} \cdot A_{16} \cdot A_{15} \cdot \overline{A_{14}} \cdot A_{13}$$

$$0xDC000 \rightarrow 0xDFFFF = A_{19} \cdot A_{18} \cdot \overline{A_{17}} \cdot A_{16} \cdot A_{15} \cdot A_{14}$$

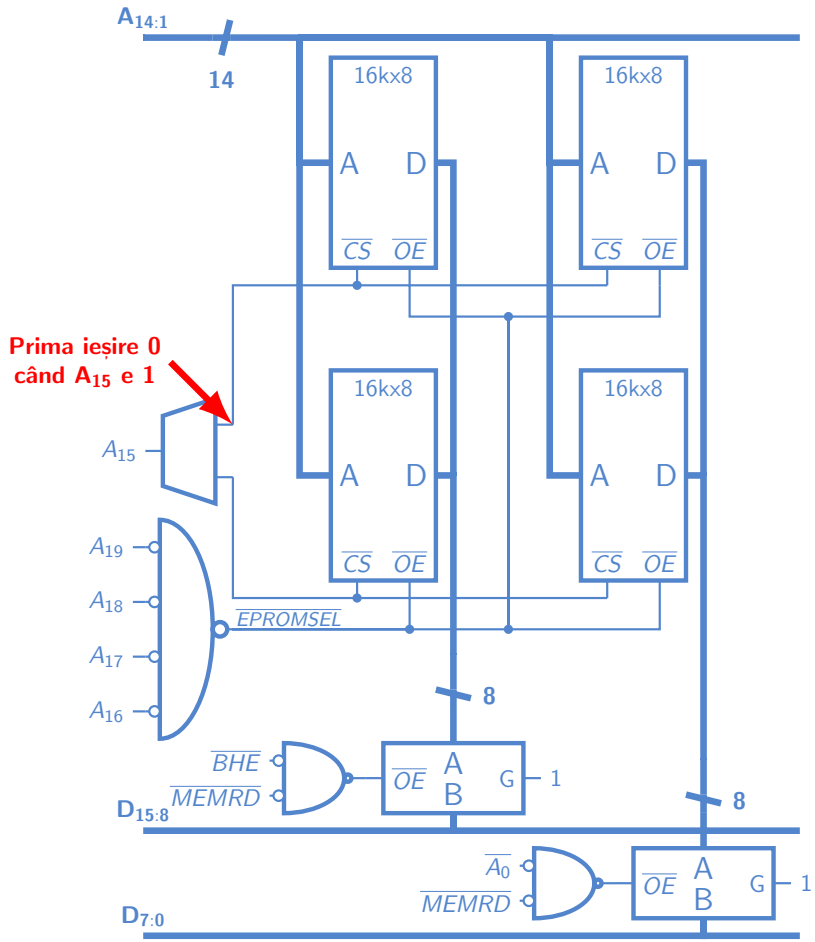
$$0xE0000 \rightarrow 0xE1FFF = A_{19} \cdot A_{18} \cdot A_{17} \cdot \overline{A_{16}} \cdot \overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}}$$


A_{19}
A_{18}
A_{17}
A_{16}
A_{15}
A_{14}
A_{13}
A_{19}
A_{18}
A_{17}
A_{16}
A_{15}
A_{14}
A_{19}
A_{18}
A_{17}
A_{16}
A_{15}
A_{14}
A_{13}
$\overline{EPROMSEL}$

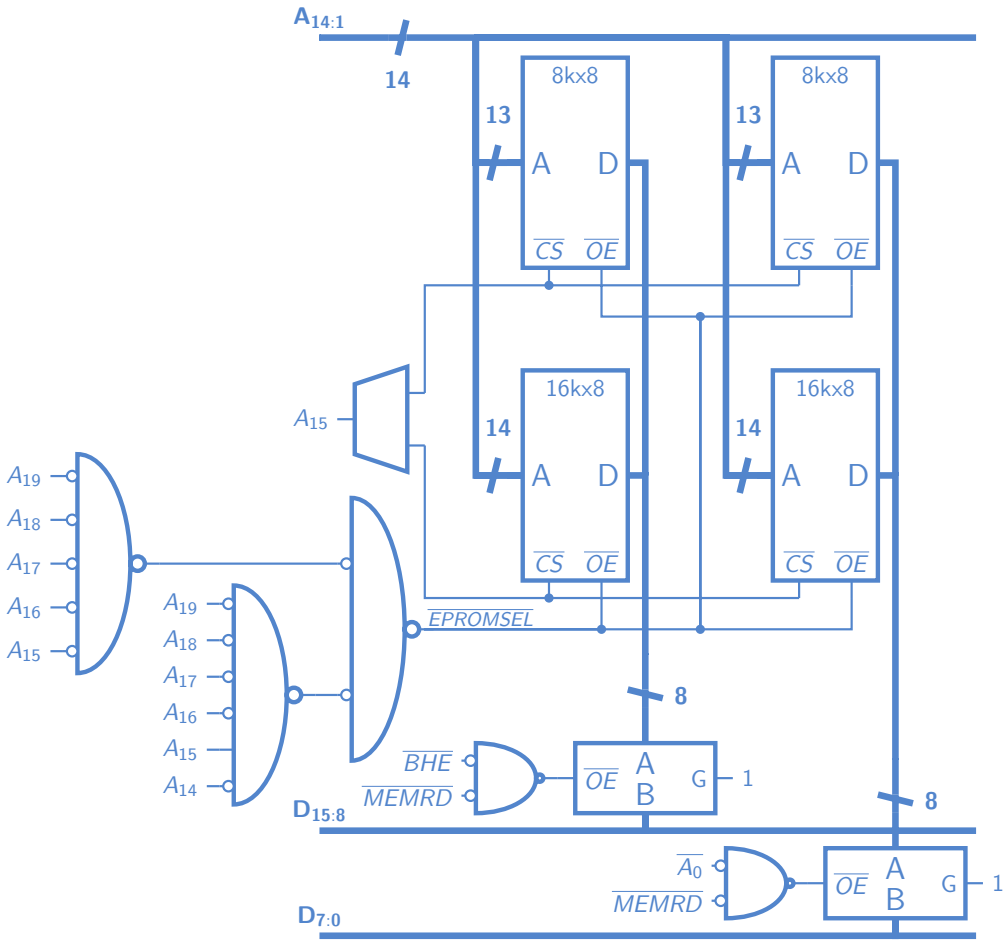
- Mapăm în primii 64K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}}}$
- *EPROMSEL* activ pe 0
- Range 0x00000–0x0FFFF



- Mapăm în primii 64K EPROM (read-only)
- Cu Chip-uri x8
- $\overline{EPROMSEL} = \overline{A_{19} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}}}$
- *EPROMSEL* activ pe 0
- Range 0x00000–0x0FFFF

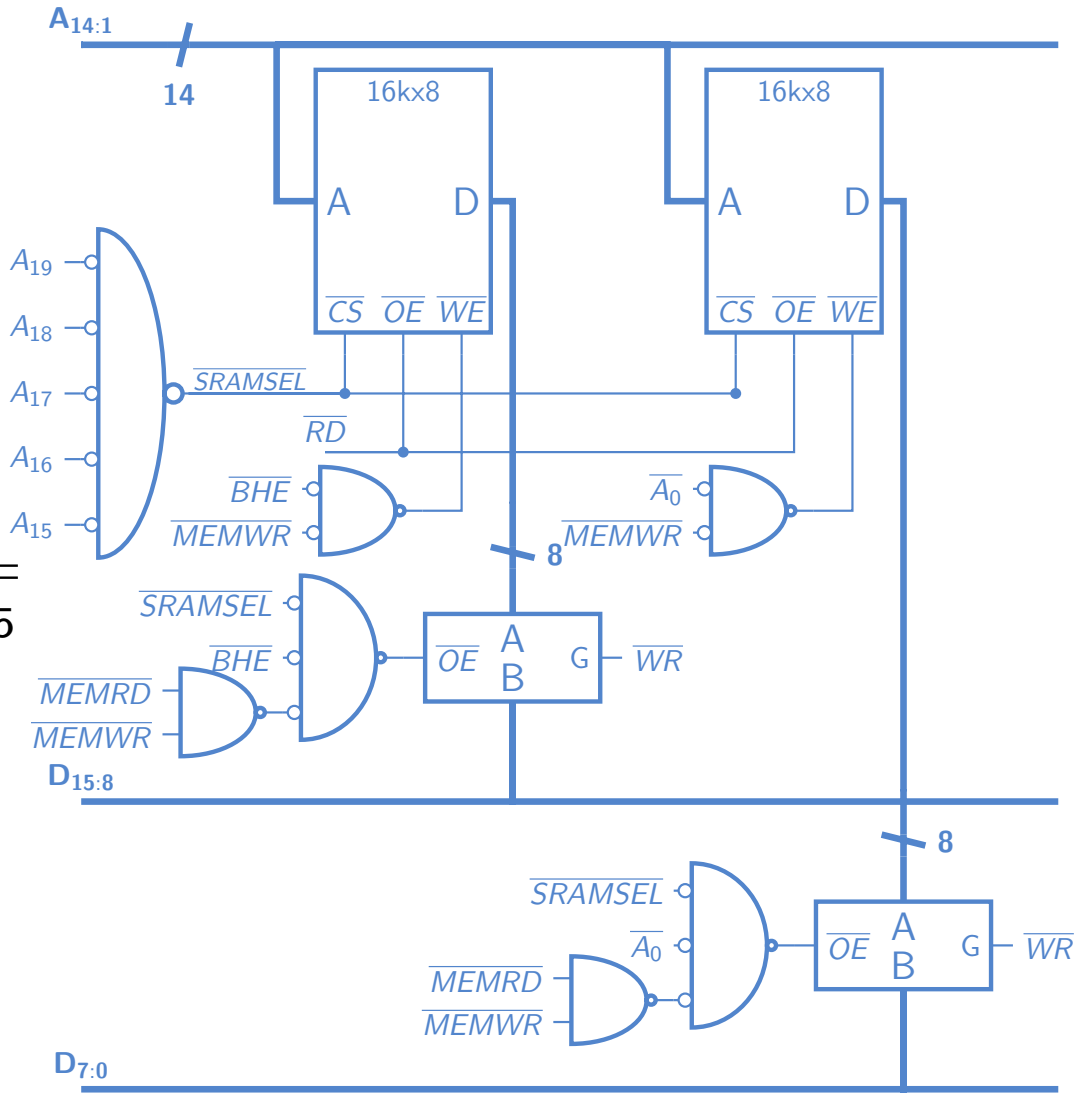


- Mapăm în primii 48K EPROM (read-only)
- Range 0x00000-0x0BFFF = 0x0000-0x07FFF + 0x08000-0x0BFFF
- $\overline{EPROMSEL} = \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}} + \overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot A_{15} \cdot \overline{A_{14}}$



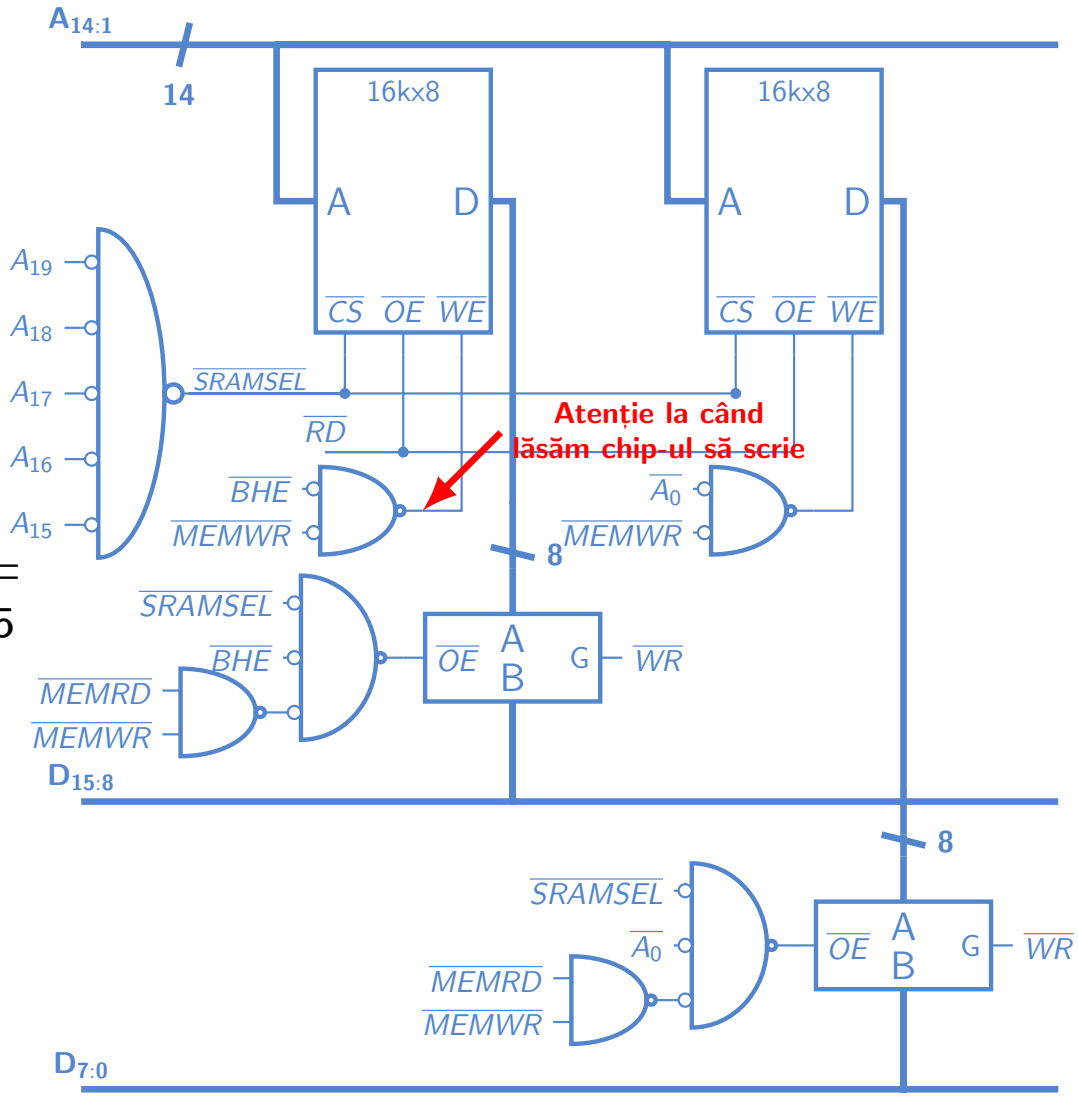
Mapare 32K SRAM

- Mapăm în primii 32K SRAM (read-write)
- Cu Chip-uri x8
- $\overline{SRAMSEL} = \overline{A_{19} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}}$
- *SRAMSEL* activ pe 0
- Range 0x00000–0x07FFF



Mapare 32K SRAM

- Mapăm în primii 32K SRAM (read-write)
- Cu Chip-uri x8
- $\overline{SRAMSEL} = \overline{\overline{A_{19}} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}}$
- *SRAMSEL* activ pe 0
- Range 0x00000–0x07FFF



- Mapăm în primii 32K SRAM (read-write)
- Cu Chip-uri x8
- $\overline{SRAMSEL} = \overline{A_{19} \cdot \overline{A_{18}} \cdot \overline{A_{17}} \cdot \overline{A_{16}} \cdot \overline{A_{15}}}$
- *SRAMSEL* activ pe 0
- Range 0x00000–0x07FFF

